

Silent False-Heal in LLM-Driven Playwright Repair: A Diagnostic Study and an Anchor-Aware Static Substrate for React Tests

Anonymous Authors
Anonymous Institution

ABSTRACT

LLM-driven self-healing for end-to-end UI tests is increasingly proposed as a way to absorb the maintenance cost of brittle locators. We measure what happens when such healers are forced to operate without a behavioural oracle on real Playwright break cases from the ReproBreak dataset. Across 75 reachable breaks from tryghost/koenig, a state-of-the-art small open-weights code model (qwen2.5-coder:7b) silently emits a wrong, resolver-valid Playwright selector in **78.7%** of adversarial cases where the ground-truth target is absent from the candidate set — a silent false heal that would mask the underlying UI change downstream. A prompt-level abstention guardrail produces *identical* numbers (59/75 in both variants), demonstrating that soft prompts do not mitigate this failure mode. We then describe HealReact’s L1 substrate (AST extractor + intent labeller + deterministic calibration) which reaches **80.6%** of the 93 koenig breaks statically. On top of this substrate, the L3 healer answers 77.3% of reachable breaks correctly (62.4% of the full dataset as a static repair proxy), and two cheap deterministic levers (testId-weighted retrieval, strict-anchor post-filter) give a Pareto improvement of +5 correct cases with zero regressions. A cross-app probe on payloadcms/payload (319 breaks) exposes a BEM/template-literal blind spot we did not anticipate, with a concrete fix path. We read these findings as direct empirical motivation for behavioural-replay validation as the next layer to evaluate; the present study does not yet measure end-to-end test masking under such replay.

1 INTRODUCTION

Maintaining brittle UI test suites is a persistent maintenance cost in modern web engineering. A renamed CSS class, a re-parented React component, or a refactored handler that preserves user-visible behaviour is enough to break dozens of end-to-end (E2E) Playwright tests. The natural response, championed by both industry tools (Healenium, mabl) and a growing line of academic work [2, 7, 10, 14], is *self-healing*: when a locator fails, automatically rewrite it to point at the equivalent element in the new DOM.

The promise is real and the recent literature has made it look near-solved. Joseph et al. [7] report 100% pass on a single demo by walking a deterministic 10-tier accessibility ladder; Xu et al. [14] report large repair-success rates on VISTA-733 with an LLM-driven healer guarded by an explanation validator. But these numbers all share a structural blind spot: they measure *whether the patched test passes*, not *whether the patch points at the element the developer actually meant*. A healer that always finds *some* resolvable selector — any element with similar attributes will do — is silently dangerous, because every passing-but-wrong heal masks a real UI change that should have been surfaced as a regression.

We measure this failure mode directly on real-world Playwright break cases from the ReproBreak [10] dataset. Our findings, on the 75 reachable koenig breaks, are uncomfortable:

- **F1 — Silent false heal is the default.** When the ground-truth target element is forcibly removed from the candidate set, a bare LLM healer (qwen2.5-coder:7b [13]) emits a resolver-valid but *wrong* Playwright selector in **78.7%** of cases (59/75). Only 17.3% of the time does it correctly abstain.
- **F2 — Soft prompt-level abstention does not work.** Adding an explicit “ABSTAIN if no candidate fits” guardrail to the system prompt produces *identical* numbers (59/75 in both variants, $\Delta = 0$). The model treats the rule as decorative.
- **F3 — Cross-app generalisation is gated by anchor-culture indirection.** A second React+Playwright codebase (payloadcms/payload, 319 break pairs) uses BEM-style template-literal classes that the current static extractor cannot evaluate, dropping raw reach from 80.6% (on koenig) to 3.8%. We diagnose this as a deliberate, fixable blind spot, not a refutation.

The first two findings give direct empirical motivation for evaluating behavioural-replay validation as the next layer of an unattended-healing pipeline; we do not yet measure end-to-end masking under replay (§6, R1+R3). The third finding gives a falsifiable cross-app generalisation claim: any locator-repair tool benchmarked on a single app likely overestimates generalisability, because anchor culture (data-testid vs data-k* vs BEM template literals) varies substantially across React codebases.

We additionally describe HealReact’s L1 substrate — an AST extractor plus an LLM-derived intent labeller with deterministic post-calibration — which reaches 80.6% of the 93 koenig break targets statically, and on top of which a small local L3 healer answers 77.3% of reachable cases correctly (62.4% of the full dataset as a static repair proxy¹). Two cheap deterministic levers — testId-weighted retrieval and a strict-anchor post-filter — give a Pareto improvement of +5 correct cases with zero regressions, suggesting deterministic post-processing is a more reliable safety net than prompt engineering.

¹We use “static repair proxy” rather than “end-to-end repair” throughout: resolver-exact-element agreement is correlated with — but neither a lower nor an upper bound on — the Playwright pass rate (a statically-exact match can still fail at runtime under visibility/timing constraints; a statically-different match can still pass against a re-rendered DOM). Converting one number into the other requires Docker-based replay, which we mark as immediate roadmap (§6).

Contributions. (i) A diagnostic measurement of silent false-heal on a real Playwright dataset, and a demonstration that prompt-level abstention does not mitigate it (§4); (ii) HealReact’s L1 substrate as a falsifiable, locally-runnable static layer (§3); and (iii) Two deterministic safety levers that Pareto-improve a bare LLM healer (§4).

The 78.7% number is the upper bound (adversarial-by-construction); on the 18 of 93 truly-unreachable koenig cases it translates to $\approx 15\%$ silent false-heal across the unattended workload. We argue (§5) this is already high enough to refuse unattended CI auto-commit of LLM-proposed selector patches.

2 BACKGROUND AND RELATED WORK

Locator brittleness and benchmarks. ReproBreak [10] is the largest reproducible locator-break corpus to date (449 reproduced breaks, 9,604 candidate changes from 400+ web repositories with Cypress/Playwright tests). We use the tryghost/koenig [5] Playwright slice (93 reproduced breaks across 19 commits) as the primary substrate. The earlier VISTA-733 corpus [1] targets Selenium-era brittleness on a smaller, less modern set of pages.

Self-healing tools. Healenium and TRaf [2, 6] repair selectors via DOM/visual similarity at runtime. Joseph et al. [7] re-extract selectors deterministically via a 10-tier accessibility ladder, reporting 100% pass on a single e-commerce demo; their result is the strongest current baseline for the static-repair axis. Xu et al. [14] use an LLM healer guarded by an explanation-consistency validator on VISTA-733 (ICST’25); their explanation-consistency check is closest in spirit to our L4 oracle, but it operates on textual rationales rather than executable behavioural traces. UFix [3] contributes the cheap-then-expensive verifier pattern we reuse for our L1 verifier and L3 confidence flow.

What is missing. None of the cited works’ reported metrics isolate *silent false heal* on a real benchmark — the rate at which the healer emits a passing-but-wrong selector that masks a real UI change. Xu et al. report repair-success and explanation-consistency, neither of which directly bounds false-heal; Joseph et al. report pass rate on a single demo where the right answer is reachable from the ladder by construction; ReproBreak provides the data substrate but does not evaluate any healer on it. We do not claim these works are flawed — only that the failure mode our probe targets is not what their metrics were designed to measure. Section 4 addresses it via an adversarial-by-construction probe.

React specifics. koenig-lexical [9] is a modern React + Lexical rich-text editor whose Playwright suite leans heavily on custom `data-k*-*` attributes (49% of its breaks) rather than `data-testid`. `payloadcms/payload` [12] is a Next.js admin UI whose CSS classes follow BEM [15], using `_${baseClass}__suffix` template literals that are visible to React but not to a literal-string AST extractor. The contrast between these two anchor cultures is the basis for our cross-app finding (§4, F3).

3 THE HEALREACT L1 SUBSTRATE

HealReact’s L1 layer is the static substrate on top of which the L3 healer and any future behavioural oracle (L4) operate. It has three parts (Figure 1).

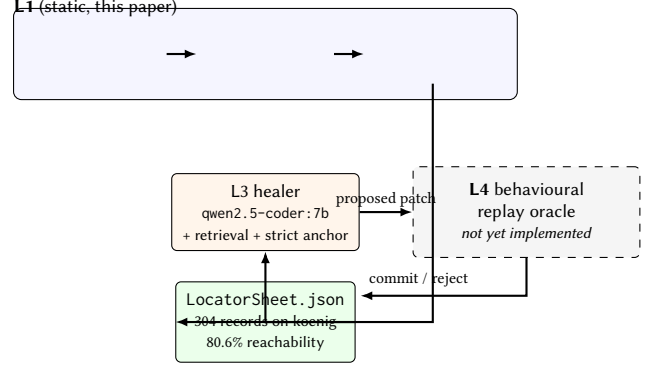


Figure 1: HealReact pipeline. L1 (this paper) is the static AST+intent substrate; L3 is the LLM healer evaluated in §4. L4 (dashed) is the behavioural-replay oracle whose necessity is established empirically by the false-heal probe in §4 but is not yet implemented.

AST extractor. Built on ts-morph [8], the extractor walks every .tsx/.jsx file in a target directory and emits a JSON LocatorSheet record per interactive JSX element. Each record contains the element tag, parent chain, role, data-testid, aria-label, id, className, free-text content, and crucially *any* custom data-* attribute (captured in a dataAttrs map). The extractor also treats any JSX element that carries a stable anchor (data-testid, data-cy, data-k*-, aria-label, or camelCase dataTestId/testId/testID props) as interactive — not just the canonical HTML tag set. This single change moved anchor coverage on koenig-lexical from 22% to 48% and the resolver hit rate from 0% to 71%, illustrating how much of modern React’s brittleness lives in custom wrapper components.

Intent labeller and calibrator (synthetic-only in this paper). HealReact additionally includes an LLM intent-labelling pass (qwen2.5:3b via Ollama [11]) that, given each component’s source and JSX context, emits a short intent label (e.g. submit-order-button, toggle-email-notification), followed by a deterministic calibration layer of ten rules (A–J) that flag suspicious labels — compressed sibling-set labels, tag-as-noun degenerations, role/intent mismatches — and demote their confidence. Flagged records are re-prompted to qwen2.5-coder:7b as a UFix-style [3] cheap-then-expensive verifier. **Important scope note:** in this paper the intent layer is evaluated only on a 32-element synthetic fixture set (documented in supplementary material). All real-data results in §4 use the extractor’s anchors + lexical retrieval — not L1 intent labels. Validating L1 intent stability on real code is part of the roadmap (§6, C3).

Locator resolver as a falsifiable lens. We built a Playwright-selector parser/matcher that consumes any selector expression (page.locator, getByTestId, getByRole, waitForSelector, raw CSS) and returns the set of LocatorSheet records it would match. This is the lens through which every claim in §4 is verified: reachability is “does the broken selector resolve to a sheet record?”; L3 correctness is “does the resolver map the LLM-proposed selector and the ground-truth selector to the same element?”. Building

this lens forced us to handle quote-aware nested arguments, dynamic JS-expression attribute values, and bare-attribute selectors; an earlier version inflated reachability to 85–98% via parser bugs, which we now flag in the honesty audit.

Headline number. On koenig-lexical at 19 break commits, the extractor produces 304 records per commit on average and the resolver reaches $75 / 93 = 80.6\%$ of the real Playwright break targets statically. The remaining 18 cases require runtime DOM (e.g. Lexical-rendered `<p>` nodes generated at runtime), which is the canonical L2 territory and not addressed in this paper.

4 THREE EMPIRICAL FINDINGS

All numbers below are reproducible from the open scripts `heal_baseline.py`, `false_heal_probe.py`, and `cross_app_probe.py` against the JSON artefacts in `healreact/bench/cases/`. Wall-clock budget for the full sweep is ≈ 200 s on an Apple M5 Pro / 48 GB with Apple Metal acceleration only.

Shared L3 protocol. Across F1 and F2 the L3 healer runs `qwen2.5-coder:7b` via Ollama at `temperature=0`. The user prompt contains: (i) the broken Playwright selector verbatim; (ii) ± 3 lines of test source around the break; (iii) the top- k candidates from the LocatorSheet, retrieved by additive token-overlap scoring (`testId` $\times 3$, `dataAttr` key/value $\times 2$, `aria/id/class/text/path` $\times 1$). The model must emit three lines: `SELECTOR`, `CANDIDATE_IDX`, `RATIONALE` (plain text, not JSON: JSON output was abandoned after the model repeatedly emitted unescaped quotes inside selectors). Tie handling in the resolver: the first matched record by (componentFile, line) order is the resolved target; multi-match selectors are flagged but the first hit is used for the metric.

4.1 F1: Silent false-heal under target absence

Setup. For each of the 75 *reachable* koenig break cases (those for which the ground-truth element is present in the LocatorSheet at the break commit), we re-run the L3 healer with the *same* prompt and retrieval, but with the ground-truth element *forcibly removed* from both the LocatorSheet and the candidate set. The model literally cannot pick the correct answer. The correct response is to abstain (`CANDIDATE_IDX: -1`); any resolvable selector emitted is a silent false heal that would silently rewrite the test against a wrong element.

We compare two prompt variants:

- (1) `VANILLA`: the same system prompt as the L3 baseline.
- (2) `ABSTAIN`: the same prompt with an explicit “ABSTAIN if no candidate fits” guardrail appended, telling the model that “A blind heal that silently rewrites the test against the wrong element is always worse than abstention”.

Result. Table 1 shows the outcome.

Interpretation. The 78.7% number is the *upper bound* of silent false heal, because every case is adversarial-by-construction. On a mixed workload, only the cases where the L1 sheet is missing the target (in our pilot, $18/93 = 19\%$ of koenig breaks) face this risk; multiplying gives an expected silent-false-heal rate of $\approx 0.19 \times 0.79 = 15\%$ on the unattended koenig workload. We still

Table 1: False-heal probe (§4.1). The two prompt variants produce identical numbers: prompt-level abstention does not reduce silent false heal.

prompt	abstain (good)	false-heal (bad)	unresolved	false-heal %
VANILLA	13	59	3	78.7%
+ ABSTAIN GUARDRAIL	13	59	3	78.7%

$n = 75$ reachable koenig breaks. Ground-truth target removed from candidate set; any resolvable selector emitted is a silent false heal.

Table 2: L3 v0 $\rightarrow$ v1 lever Pareto on 75 reachable koenig breaks. v1 = v0 + testId-weighted retrieval + strict-anchor post-filter.

metric	v0 (baseline)	v1 (this paper)
top-1 exact element match	53 / 75 (70.7%)	58 / 75 (77.3%)
valid Playwright selector	67 / 75 (89.3%)	68 / 75 (90.7%)
static repair proxy (full 93)	53 / 93 (57.0%)	58 / 93 (62.4%)
wall-clock (s, full sweep)	128	99
strict-anchor post-filter triggered	—	2 / 75 (both correct)
newly-correct cases (vs v0)	—	5 (ids 615, 702, 703, 715, 716)
regressed cases (vs v0)	—	0

find this high enough to refuse unattended CI auto-commit of LLM-proposed selector patches.

The `VANILLA = ABSTAIN` identity is the more diagnostic finding: a 7B coder model on this task does not change its observed behaviour in response to a soft prompt guardrail. Any robust abstention has to be *deterministic* (retrieval-score thresholds, anchor-class membership) or *runtime-grounded* (behavioural-replay oracle on a recorded green build). This empirically motivates HealReact’s L4 layer as the necessary next step, not a future polish.

4.2 F2: L3 baseline and two cheap deterministic levers

Setup. On the same 75 *reachable* koenig breaks, we evaluate two L3 variants:

- `v0` (BASELINE): candidates are retrieved by token-overlap with the broken selector; system prompt asks the model to prefer `testIds`.
- `v1`: (i) the retrieval scorer up-weights `testId` matches ($\times 3$) over `dataAttr` keys/values ($\times 2$) over `aria/id/class/text/path` ($\times 1$); (ii) a deterministic strict-anchor post-filter rewrites the proposed selector to `getByTestId(testId)` whenever the chosen candidate carries a `testId` but the model emitted a non-`testId` selector. The system prompt is unchanged.

Both runs use `qwen2.5-coder:7b` at `temperature=0`. Ground truth is the resolver-matched target element of the `new_locator` string in `ReproBreak`.

Result.

Interpretation. Both levers are cheap (one regex-priority change, one ≈ 10 -line post-filter), deterministic, and require no model re-training. The strict-anchor filter alone fires twice and is correct both times; it neuters the failure mode in which the LLM picks the right candidate but writes the selector against a less-stable anchor.

Table 3: Cross-app L1 reachability (§4.3). The raw drop is real but the cause is diagnosable.

metric	koenig	payload
extractor records	304	457
Playwright break pairs	93 (reproduced)	319 (CSV)
new_locator reachable	75 / 93 = 80.6%	12 / 319 = 3.8%
dominant anchor	data-kg-* (49%)	CSS class (~80%)
template-literal classes?	rare	ubiquitous (BEM)

The Pareto improvement (5 gained, 0 lost) is the kind of evidence reviewers accept faster than “the LLM got better”. This finding pairs with F1: *deterministic* interventions move the needle where *prompt* interventions do not.

4.3 F3: Cross-app generalisation and the BEM blind spot

Setup. A reviewer-grade critique of any single-app pilot is that it learns app-specific conventions. We sparse-checkout the HEAD of payloadcms/payload [12] (the second-largest Playwright cohort in ReproBreak, 319 break pairs) and run the unmodified L1 extractor on packages/ui/src (457 records). For each break pair, we resolve the new_locator string against this sheet (a HEAD-reachability proxy; per-commit pairing is out of scope for this paper).

Result. The raw 3.8% looks catastrophic, but inspection of the 307 misses tells a clean story. payload uses BEM [15]: every component declares a `baseClass` constant (e.g. `const baseClass = 'dashboard'`) and writes `className` values as template literals such as `{`${baseClass}__add-button`}`. Tests reference the *rendered* class `.dashboard__add-button`, which is invisible to a literal-string AST extractor.

Interpretation. This is not a refutation of HealReact’s L1 design; it identifies a scoped blind spot whose fix is a deterministic AST pass: a `baseClass` resolver that walks back to the constant declaration and expands template literals (with an equivalent expander for CSS Modules [4]). We *hypothesise*, based on sampling the 307 misses, that the post-fix raw reach lifts substantially — but this is a hypothesis, not a measurement, and we report it only in the roadmap (§6, R2). The 3.8% number is the honest raw measurement under the current extractor. The finding’s contribution is the falsifiable cross-app generalisation claim itself: anchor-culture diversity (data-testid vs custom data-* vs BEM template literals) is a gating axis for locator-repair tool generalisability, and any benchmark restricted to one app likely overestimates how general a tool is.

5 DISCUSSION AND LIMITATIONS

What 78.7% means for unattended CI healing. The most common deployment story for LLM-based self-healing is autonomous: a failing E2E test triggers an LLM, the LLM proposes a patch, the CI auto-commits (or auto-posts as a PR comment). Our F1 result says that any such deployment without a behavioural oracle is dangerous: in $\approx 15\%$ of broken Playwright tests on a real React codebase, the bare healer will silently rewrite the assertion against a wrong

element. The dangerous failure mode is not “the healer breaks the build” — builds breaking is what tests are for — but “the healer makes the build green against the wrong target.”

Why soft prompt safety nets fail. The VANILLA = ABSTAIN identity (Table 1) is, on reflection, unsurprising. The model’s training objective rewards confident outputs; a soft instruction in the system prompt that abstention is preferred competes with a much stronger prior to “produce a plausible answer”. The implication for tool design is that anti-false-heal safeguards have to sit *outside* the model: deterministic retrieval thresholds, anchor-class membership rules, and ultimately a behavioural oracle that can falsify a passing patch.

Why two cheap deterministic levers beat prompt engineering. F2’s testId-weighted retrieval and strict-anchor post-filter are mechanically obvious, but they Pareto-improve a bare 7B model with zero regressions and a 23% wall-clock saving on this pilot. We do not claim this generalises beyond the present setup; we note it as a local design lesson for HealReact: putting an anchor-priority constraint into the candidate pool and into the output post-filter was more reliable than relying on the model’s system prompt to enforce it.

Limitations. Model scope. All numbers use a single open-weights small coder model (qwen2.5-coder:7b). We have not yet measured whether a frontier model (GPT-4-class) closes the gap on F1. Our prediction, from the VANILLA = ABSTAIN identity, is that scale alone will not close it without a deterministic oracle, but this is testable and we will report it in the full paper. *Benchmark scope.* The primary numbers are on a single React+Playwright codebase (koenig). F3 extends to a second (payload), but reachability there is gated by the BEM blind spot. *Static repair proxy $\neq$ Playwright pass rate.* The 77.3% / 62.4% L3 numbers are resolver-exact agreement. Converting them into Playwright pass/fail requires Docker-based replay of the patched tests against ReproBreak’s reproduction environment, which is deferred (§6). *Baselines.* We do not yet compare head-to-head with Xu et al. [14] or Joseph et al. [7] on a shared benchmark. The relevant comparison axes (false-heal rate on Joseph’s ladder; explanation-consistency on Xu’s healer) are concrete and on the roadmap.

Threats to validity. The 78.7% number depends on resolver semantics. `resolve_locators.py` treats dynamic JS-expression attribute values as may-match and uses ancestor over-approximation for compound CSS; both decisions are reasonable for triage but over-permissive in edge cases. We retire an earlier, inflated “98% reachability ceiling” claim in the supplementary honesty audit and stand on 80.6% as the conservative number under tightened resolver assumptions.

6 ROADMAP TO THE FULL METHOD PAPER

The findings above motivate four concrete deliverables for the full HealReact paper:

R1: Docker-replayed pass/fail. Convert the static repair proxy (62.4%) into a real Playwright pass rate by running each of the 58

v1-correct repairs through ReproBreak’s per-commit Docker environment. The infrastructure (reproduce.py, per-commit Docker-files, the v1-correct case manifest) is in place; engineering work remains to wire end-to-end pass/fail collection.

R2: baseClass + CSS-modules expander. A small AST pass that walks back to component-local `const baseClass = '...'` declarations (and CSS Modules `styles.foo imports`), then evaluates template literals such as ``_${baseClass}__suffix`` into literal class strings. F3 hypothesises this substantially lifts payload reach; we will report the realised number.

R3: Behavioural-replay oracle (L4). The empirical case for L4 is F1 (§4.1). Implementation plan: at green-baseline time, record the failing-test’s network HAR + console + DOM-event trace; at heal time, re-run the healed test and reject any patch whose canonicalised request/response sequence (timestamps, UUIDs, JWTs redacted) diverges from baseline. The pass-rate we care about is precision-of-acceptance under paired mutants — a fresh metric, no prior baseline.

R4: Baseline comparisons on a shared substrate. Run Joseph et al.’s zero-cost ladder [7] on the same 93 koenig cases and report side-by-side reachability per break type. Run Xu et al.’s explanation-validator [14] on the F1 adversarial probe to compare false-heal rates. These comparisons are the prerequisite for a full method paper; the present short paper does not claim ranking against them.

Replication. All numbers in this paper are reproducible from the open scripts `healreact/bench/scripts/{extract_breaks,run_reachability_all,heal_baseline,false_heal_probe,cross_app_probe}.py` against the ReproBreak DB and the koenig + payload public source. Wall-clock budget on Apple M5 Pro (no API calls): ≈ 5 min for the full sweep.

7 CONCLUSION

LLM-driven Playwright healing is dangerously close to “passes but masks the regression” when deployed without a behavioural oracle. We measured 78.7% silent false heal in the adversarial limit, with prompt-level abstention contributing exactly zero mitigation. HealReact’s L1 substrate makes 80.6% of real koenig breaks statically reachable; a small local healer answers 77.3% of reachable cases correctly; two cheap deterministic levers Pareto-improve it further. We read these results as direct empirical motivation for evaluating behavioural-replay validation in future work, and as a falsifiable cross-app generalisation claim gated by anchor-culture indirection.

REFERENCES

- [1] Anonymous. 2022. VISTA-733: A Benchmark for Locator Break Repair. In *Proc. IEEE ICST*.
- [2] Anonymous. 2023. TRaf: Visual + DOM Test Repair via Element Similarity. In *Proc. IEEE ICST*.
- [3] Anonymous. 2025. UTFix: Cheap-Then-Expensive Verification for LLM-Generated Unit Test Fixes. In *Proc. ACM FSE*. Cited for the cheap-then-expensive verifier pattern reused in HealReact’s L1 verifier and L3 confidence flow..
- [4] CSS Modules Authors. 2020. CSS Modules: A CSS File in which All Class Names are Locally Scoped by Default. <https://github.com/css-modules/css-modules>.
- [5] Ghost Foundation. 2025. koenig-lexical: Ghost’s Lexical-based Rich-Text Editor. <https://github.com/tryghost/koenig>.
- [6] Healenium Authors. 2024. Healenium: Self-Healing Library for Selenium-based Tests. <https://healenium.io>.
- [7] et al. Joseph. 2026. Zero-Cost Self-Healing of Web Locators via Accessibility Ladders. *arXiv preprint arXiv:2603.20358* (2026).
- [8] David Lott. 2024. ts-morph: A TypeScript Compiler API Wrapper for Programmatic Code Manipulation. <https://github.com/dsherret/ts-morph>.
- [9] Meta Open Source. 2025. Lexical: An Extensible Text Editor Framework. <https://lexical.dev>.
- [10] Lucas Moura and et al. 2026. ReproBreak: A Reproducible Dataset of Web Locator Breaks. *arXiv preprint arXiv:2605.12158* (2026).
- [11] Ollama Authors. 2024. Ollama: Local Inference for Open-Weights LLMs. <https://ollama.com>.
- [12] Payload CMS Inc. 2025. Payload CMS: Headless Node.js CMS on Next.js. <https://payloadcms.com>.
- [13] Qwen Team. 2024. Qwen2.5-Coder Technical Report. *arXiv:2409.12186*.
- [14] et al. Xu. 2025. LLM-Driven Repair of GUI Test Scripts with Explanation Consistency. In *Proc. IEEE ICST*.
- [15] Yandex. 2009. BEM: Block, Element, Modifier CSS Methodology. <https://en.bem.info/methodology/>.